DataSci 306 Final Project

DataSci 306 Instructional Team

2026-04-28

Investigating the Internet Movie Database (IMDB)

The Internet Movie Database (IMDb) contains information on millions of movies and television programs. They offer several non-commercial use datasets (documentation link). For this project we will analyze a **sample** of 100,000 titles from the IMDBb.

Part I: Preprocessing

- Edit your `.gitignore` file to ignore all files with the `.rda` extension. (Add and commit)
- Create a new file in the `data/` directory called "Preprocessing.Rmd". The remaining instructions in this section are to be completed in that file.
- Write a function that will load a table from the IMDb files in the `data/` directory.
  - The function should take the file name (without the ".csv.gz" portion) as an argument
  - The function should load the appropriate `.csv.gz` file.
  - Make sure that all "\N" values (which IMDB uses to indicate missing values) are turned into proper NA values in R
  - The function should return the table.
- For each of the `.csv.gz` files, use your function to load the table, then save it into a variable (e.g. `name_basics <- preprocess("name_basics")`) and use the `write_rds` function (e.g., `write_rds(name_basics, "name_basics.rda")`.
- Run the function on all of the `*_sample.csv.gz` files to created processed `.rda` files.
- In your other files, you can load these using the `TABLE <- read_rds("data/FILENAME.rda")` function.

```
NAME_BASICS <- read_rds("data/name_basics.rda")
TITLE_BASICS <- read_rds("data/title_basics.rda")
TITLE_PRINCIPALS <- read_rds("data/title_principals.rda")
TITLE_RATINGS <- read_rds("data/title_ratings.rda")
```

Part II: EDA of individual tables

- For each of the 4 tables, perform basic exploratory data analysis. Report the following information:
  - For each quantitative column, provide some summary statistics
  - For any character columns, decide if they are actually representing factors/categorical data with a moderate number of columns. If so report the distributions for these variables.
  - Provide a plot for each table. Across all of the plots, try to show off the most possible different ggplot features (`geoms_` functions, `stat_` functions, coordinate systems, facets, use of several variables, annotations)

```
NAME_BASICS %>% select(where(is.numeric)) %>% summary()
```

```
##    birthYear        deathYear
##  Min.   :  37     Min.   :  44
##  1st Qu.:1933     1st Qu.:1980
##  Median :1959     Median :2000
```

```
##  Mean   :1953     Mean   :1994
##  3rd Qu.:1976     3rd Qu.:2014
##  Max.   :2021     Max.   :2024
##  NA's   :337769   NA's   :446685
```

```
TITLE_BASICS %>% select(where(is.numeric)) %>% summary()
```

```
##     isAdult            startYear       endYear
##  Min.   :0.000e+00   Min.   :1887   Min.   :1938
##  1st Qu.:0.000e+00   1st Qu.:1997   1st Qu.:2001
##  Median :0.000e+00   Median :2011   Median :2013
##  Mean   :3.602e-02   Mean   :2003   Mean   :2008
##  3rd Qu.:0.000e+00   3rd Qu.:2018   3rd Qu.:2019
##  Max.   :2.020e+03   Max.   :2025   Max.   :2025
##                      NA's   :17     NA's   :96408
```

```
TITLE_PRINCIPALS %>% select(where(is.numeric)) %>% summary()
```

```
##     ordering
##  Min.   : 1.00
##  1st Qu.: 4.00
##  Median : 8.00
##  Mean   : 9.06
##  3rd Qu.:13.00
##  Max.   :62.00
```

```
TITLE_RATINGS %>% select(where(is.numeric)) %>% summary()
```

```
##  averageRating       numVotes
##  Min.   : 1.000   Min.   :      5.0
##  1st Qu.: 6.200   1st Qu.:     11.0
##  Median : 7.200   Median :     26.0
##  Mean   : 6.963   Mean   :    979.9
##  3rd Qu.: 7.900   3rd Qu.:    101.0
##  Max.   :10.000   Max.   :2279226.0
```

```
TITLE_PRINCIPALS$category <- as.factor(TITLE_PRINCIPALS$category)
TITLE_BASICS$titleType <- as.factor(TITLE_BASICS$titleType)
summary(TITLE_PRINCIPALS$category)
```

```
##               actor             actress     archive_footage       archive_sound
##              406747              242512                7935                 267
##    casting_director     cinematographer            composer            director
##               32853               60902               61829               87017
##              editor            producer production_designer                self
##               77280               98111               28164              111477
##              writer
##              146370
```

```
summary(TITLE_BASICS$titleType)
```

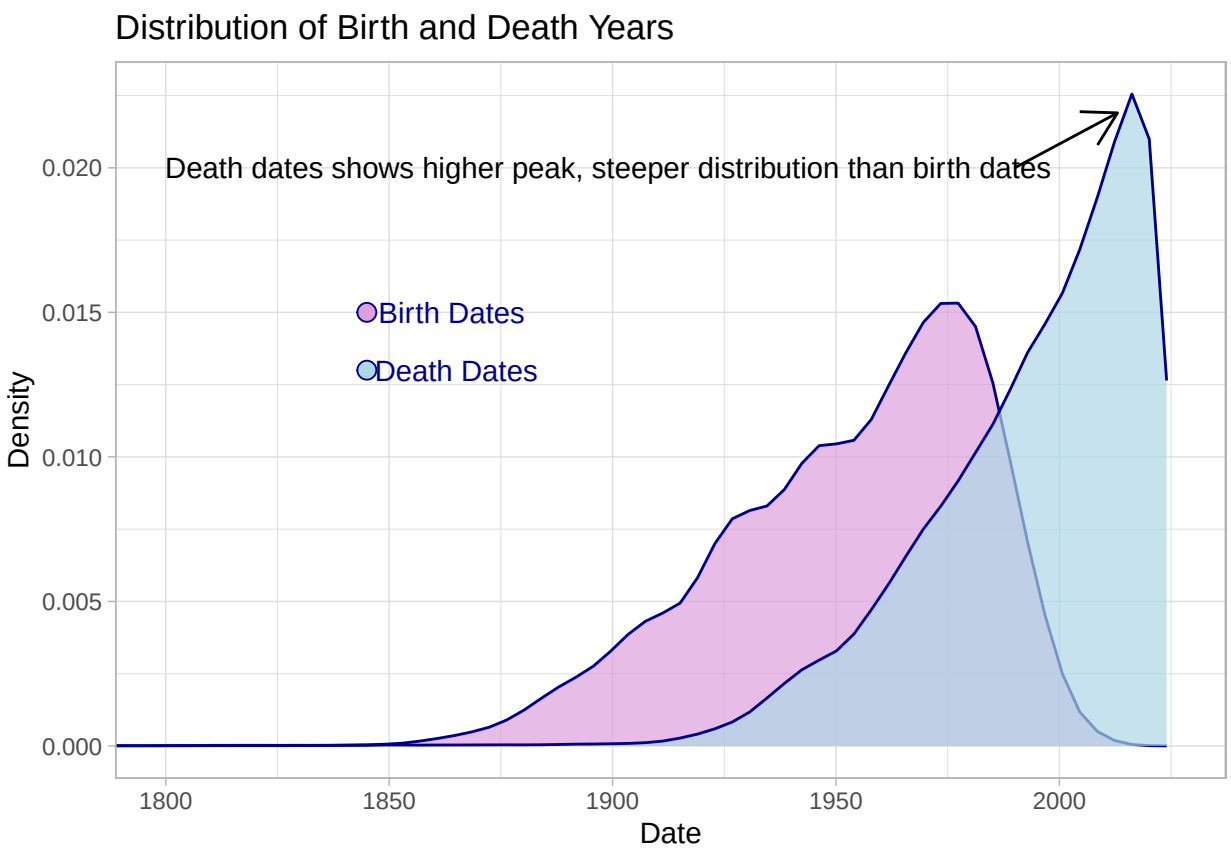
```
##        movie        short    tvEpisode tvMiniSeries      tvMovie     tvSeries
##        21467        11118        50194         1127         3703         6555
##      tvShort    tvSpecial        video    videoGame
##          161          837         3617         1221
```

```
ggplot(data = NAME_BASICS) +
  geom_density(aes(x = birthYear), fill = "plum", color = "darkblue", alpha = .7) + coord_cartesian(xlim
```

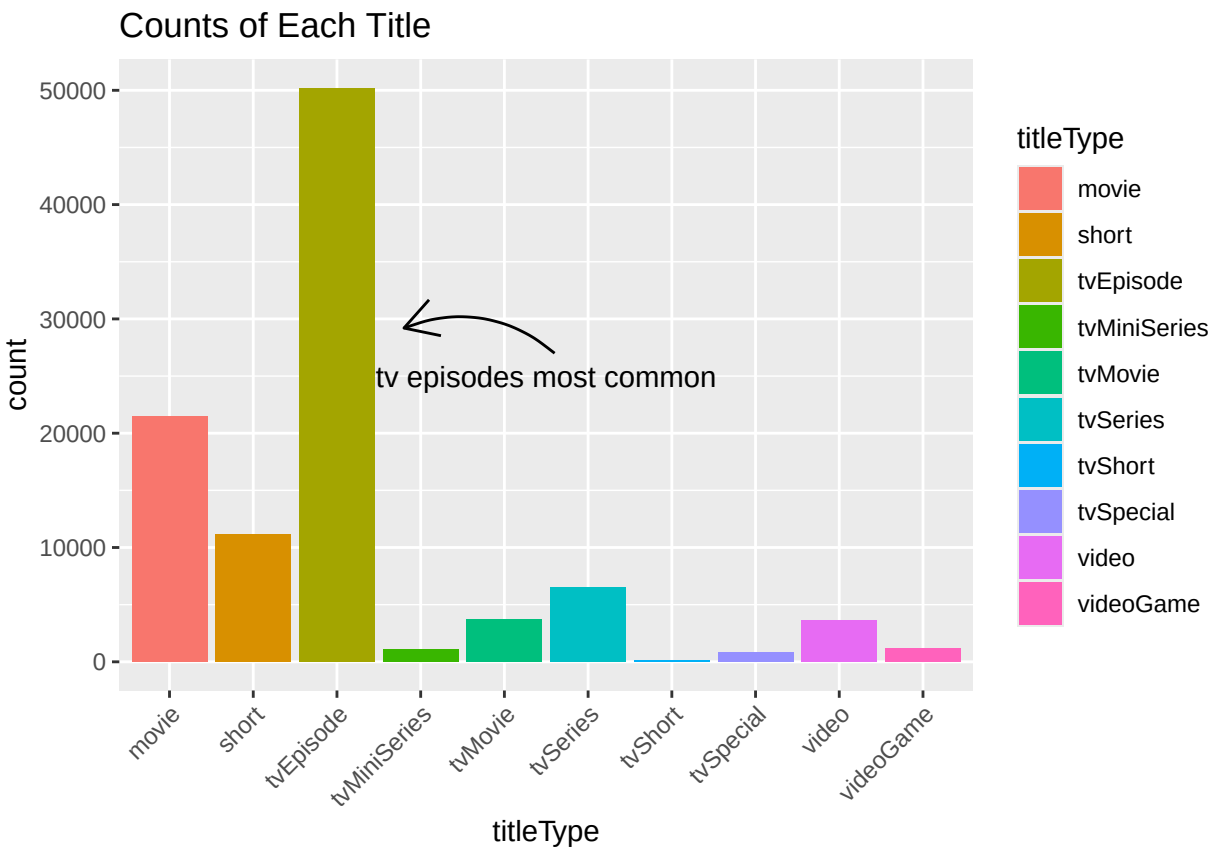
```
  geom_density(aes(x = deathYear), fill = "lightblue", color = "darkblue", alpha = .7) +
 annotate("text", x = 1899, y = .02, label = "Death dates shows higher peak, steeper distribution than b
  labs(x = "Date", y = "Density", title = "Distribution of Birth and Death Years") +
  annotate("point", x = 1845, y = .015, fill = "plum", color = "darkblue", shape = 21, size = 3) +
  annotate("point", x = 1845, y = .013, fill = "lightblue", color = "darkblue", shape = 21, size = 3) +
  annotate("text", x = 1864, y = .015, color = "darkblue", label = "Birth Dates", size = 4) +
  annotate("text", x = 1865, y = .013, color = "darkblue", label = "Death Dates", size = 4) +
  annotate("segment", x = 1990, y = .02, xend = 2013, yend = .0219, arrow = arrow(length = unit(.5, "cm
```

```
## Warning: Removed 337769 rows containing non-finite outside the scale range
## (`stat_density()`).
```

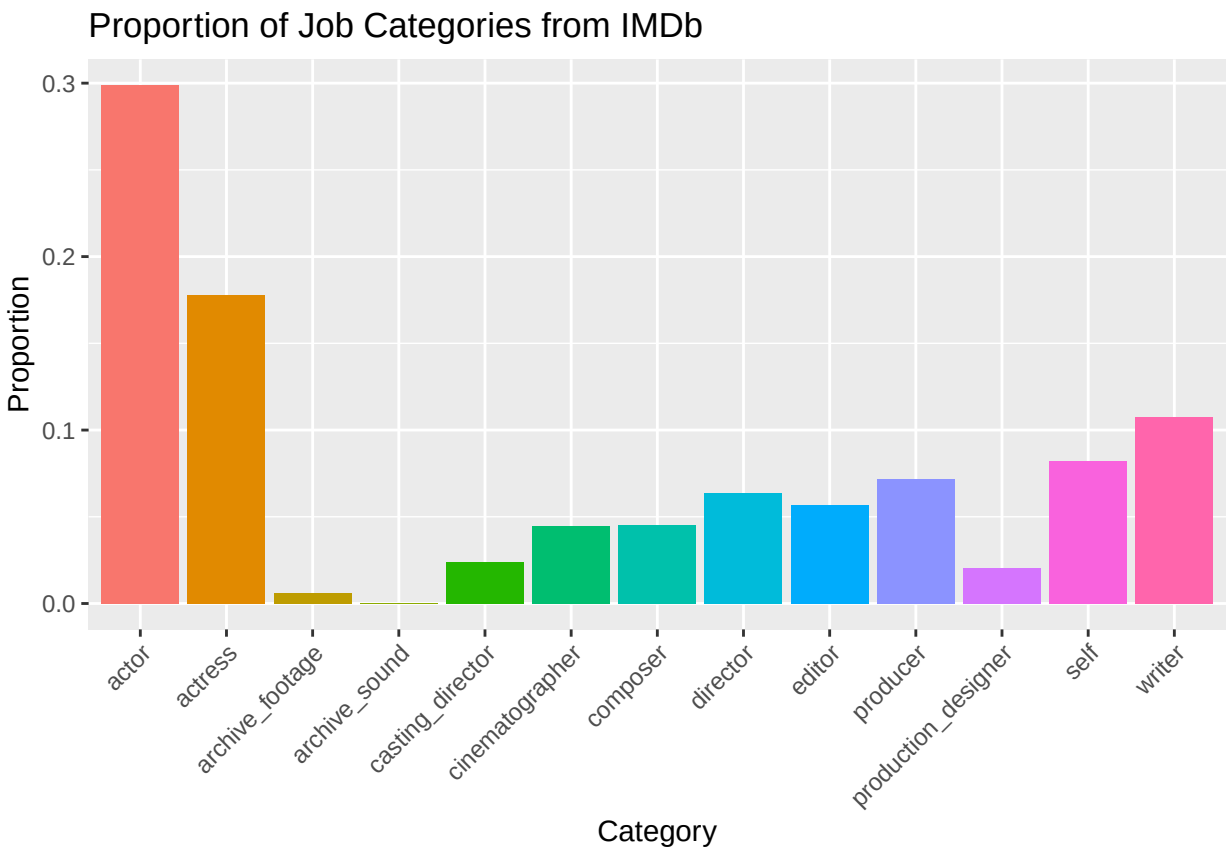
```
## Warning: Removed 446685 rows containing non-finite outside the scale range
## (`stat_density()`).
```




```
ggplot(data = TITLE_BASICS) +
  geom_bar(aes(x = titleType, fill = titleType)) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1)) +
  annotate("text", x = 5.5, y = 25000, label = "tv episodes most common") +
labs(title = "Counts of Each Title") +
  annotate("curve", x = 5.6, y = 27000, xend = 3.8, yend = 29200, curvature = .3, arrow = arrow(length =
```

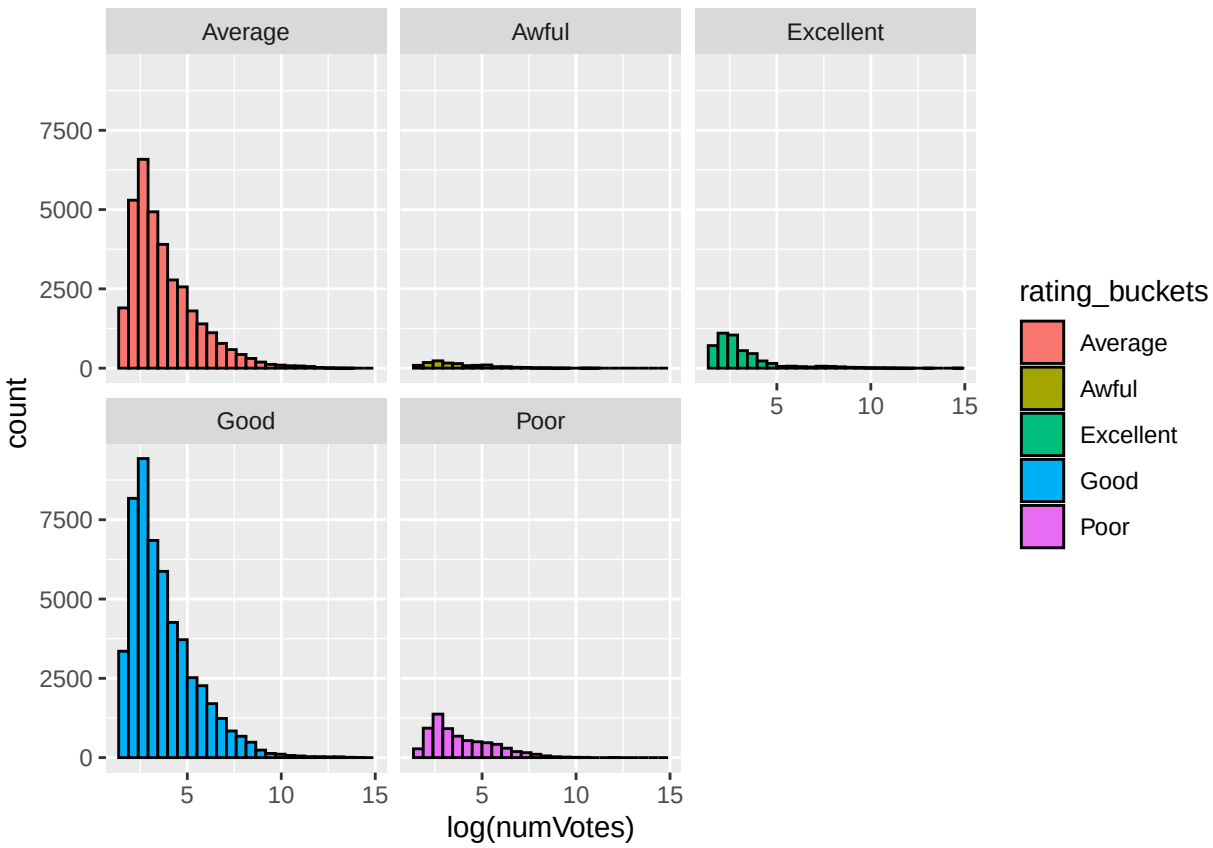



```r
ggplot(data = TITLE_PRINCIPALS) +
  geom_bar(aes(x = as.factor(category), y = after_stat(prop), group = 1,
               fill = after_stat(as.factor(x))), show.legend = FALSE) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1)) +
  labs(x = "Category", y = "Proportion", title = "Proportion of Job Categories from IMDb")
```




```
TITLE_RATINGS <- TITLE_RATINGS %>% mutate(rating_buckets = case_when(
  averageRating < 3 ~ "Awful",
  averageRating < 5  ~ "Poor",
  averageRating < 7 ~ "Average",
  averageRating < 9 ~ "Good",
  TRUE ~ "Excellent"
))


ggplot(data = TITLE_RATINGS) +
  geom_histogram(aes(x = log(numVotes),
                     fill = rating_buckets), color = "black", bins = 26) +
  facet_wrap(~as.factor(rating_buckets))
```



- For the `titles_basics` table
  - use two different variables to group and explore how `runtimeMinutes` varies for these different groups. Produce appropriate summaries.
  - How many titles are known for name that is different than the original release name? 7244 titles have a primary title and original title that do not match
  - Graph the conditional distributions of release year based on the previous results. Comment on any trends you observe.

The titles that are known for a name that is the same as the original release name have higher density in more recent years. The titles that are known for a name that is different than the original release name have higher density in years between 1950 - 2000.

```
#create groups for start year

TITLE_BASICS <- TITLE_BASICS %>% mutate(two_decades =
  cut(startYear, breaks = c(1887, 1899, 1919, 1939, 1959, 1979, 1999, 2025),
    labels = c("Before 1900","1900_1919", "1920_1939","1940_1959", "1960_1979", "1980_1999", "2000_2025
  )
)

#use two_decade and title type to explore run time

TITLE_BASICS %>% summarize(mean_runtime = mean(as.numeric(runtimeMinutes), na.rm = TRUE), .by = c(two_de
```

```
## Warning: There was 1 warning in `summarize()`.
## i In argument: `mean_runtime = mean(as.numeric(runtimeMinutes), na.rm = TRUE)`.
## i In group 45: `two_decades = NA`, `titleType = tvEpisode`.
## Caused by warning in `mean()`:
```

```
## ! NAs introduced by coercion
##    two_decades    titleType mean_runtime
## 1  Before 1900        short     1.147541
## 2    1900_1919        short    10.451178
## 3    1900_1919        movie    77.875000
## 4    1920_1939        movie    80.811659
## 5    1920_1939        short    12.621315
## 6    1940_1959        movie    89.254860
## 7    1940_1959        short    12.207983
## 8    1940_1959     tvSeries    50.524590
## 9    1940_1959    tvEpisode    33.125000
## 10   1940_1959      tvMovie    62.233333
## 11   1940_1959      tvShort    14.600000
## 12   1960_1979        movie    93.967271
## 13   1960_1979 tvMiniSeries   197.923077
## 14   1960_1979     tvSeries    68.285714
## 15   1960_1979      tvMovie    81.393120
## 16   1960_1979    tvEpisode    41.496088
## 17   1960_1979        short    13.720887
## 18   1960_1979      tvShort    13.176471
## 19   1980_1999        movie    99.022925
## 20   1980_1999        short    16.699015
## 21   1980_1999    tvEpisode    37.917670
## 22   1980_1999     tvSeries    50.350245
## 23   1980_1999      tvMovie    88.084076
## 24   1980_1999 tvMiniSeries   184.539683
## 25   1980_1999        video    73.529006
## 26   1980_1999    videoGame    66.000000
## 27   1980_1999    tvSpecial   100.432990
## 28   1980_1999      tvShort    14.170732
## 29   2000_2025        movie    94.567419
## 30   2000_2025        short    14.543391
## 31   2000_2025    tvEpisode    38.960949
## 32   2000_2025      tvMovie    78.455992
## 33   2000_2025        video    70.215859
## 34   1940_1959 tvMiniSeries    25.500000
## 35   1960_1979    tvSpecial    68.060606
## 36   2000_2025     tvSeries    46.580343
## 37   2000_2025    videoGame   180.875000
## 38   2000_2025 tvMiniSeries    79.349425
## 39   2000_2025    tvSpecial    89.007519
## 40   2000_2025      tvShort    11.292683
## 41   1960_1979        video    36.428571
## 42   1920_1939      tvMovie    60.000000
## 43   1940_1959    tvSpecial    80.000000
## 44   1960_1979    videoGame          NaN
## 45        <NA>    tvEpisode    40.600000
## 46        <NA>        short     2.333333
## 47        <NA>        movie    81.000000
## 48   1920_1939     tvSeries    15.000000
## 49        <NA>      tvMovie          NaN
## 50        <NA>     tvSeries          NaN
```

```r
TITLE_BASICS %>% summarize(sum(primaryTitle != originalTitle))
```

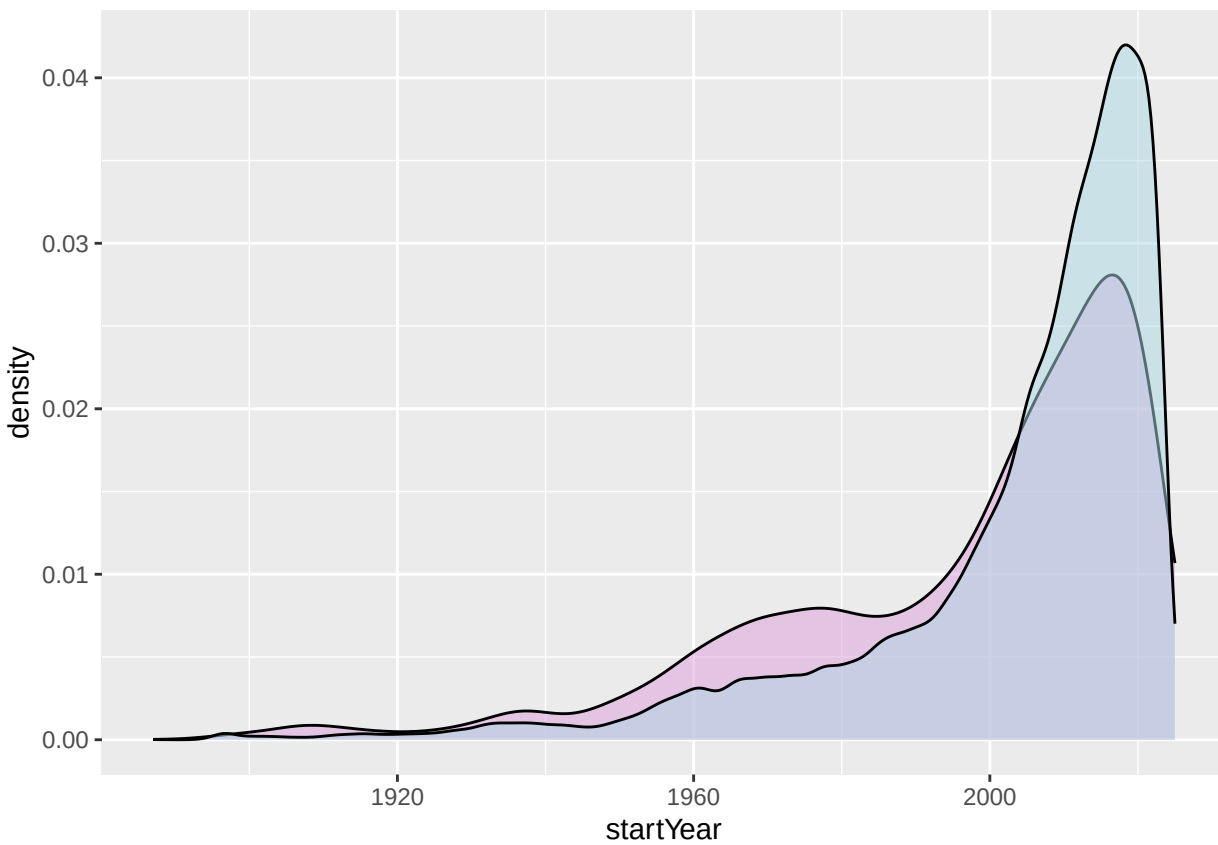
```
##   sum(primaryTitle != originalTitle)
## 1                               7244
```

```r
no_match <- TITLE_BASICS %>% filter(primaryTitle != originalTitle)
match <- TITLE_BASICS %>% filter(primaryTitle == originalTitle)

ggplot() +
    geom_density(data = no_match, aes(x = startYear),
                 color = "black", fill = "plum", alpha = .5) +
  geom_density(data = match, aes(x = startYear),
               color = "black", fill = "lightblue", alpha = .5)
```

```
## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_density()`).
```

```
## Warning: Removed 16 rows containing non-finite outside the scale range
## (`stat_density()`).
```



- For the ratings, use the `cut` function to break the data into three groups based on the average ratings. Are higher rated titles rated more often or less often than lower rated titles?

Higher rated titled are rated more often than lower rated titles.

- For the names table,
  - Count the number of titles each person is known for and plot this distribution.
  - investigate the age of cast members
    - Group the data into living and deceased cast members. - distribution centered around 100

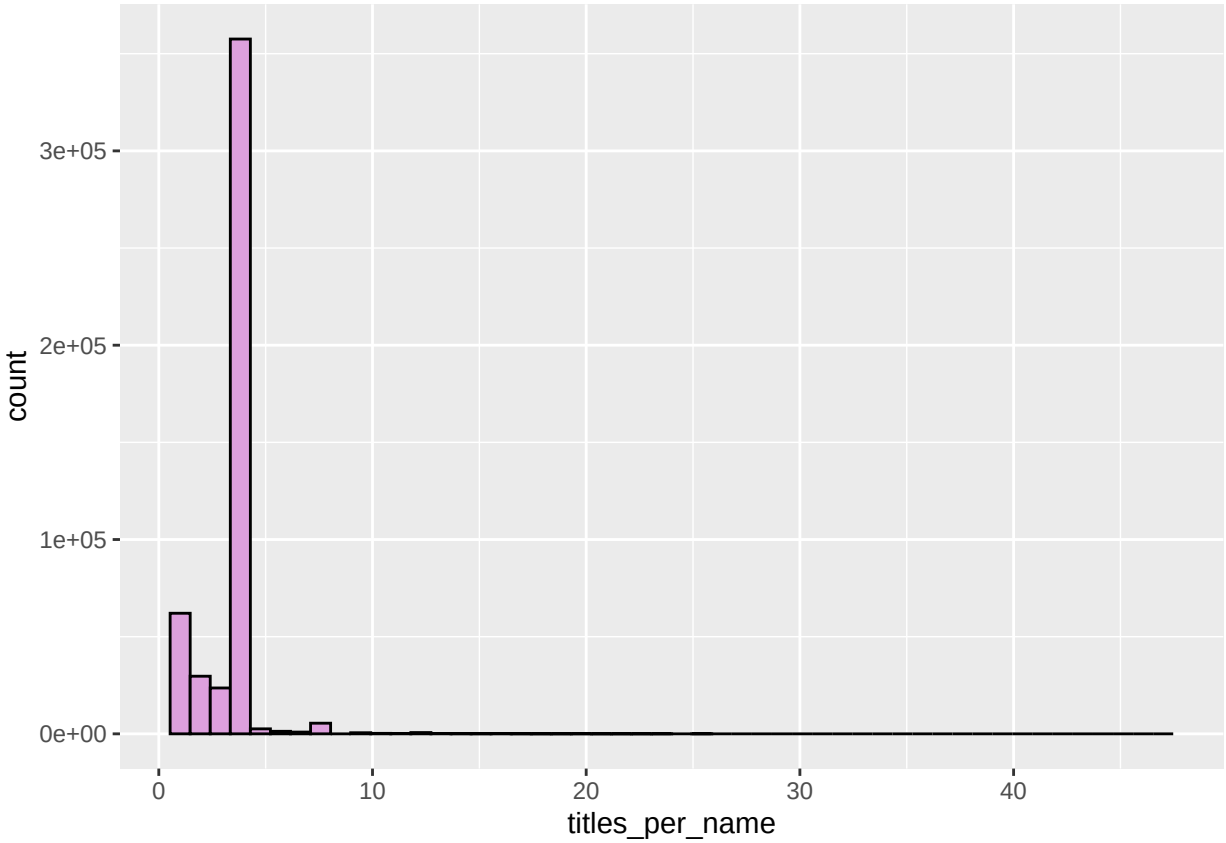
years
* For deceased cast members, provide a graph that shows the distribution of ages.
* Do the same for living cast members. - distribution centered around 55 years

```
TITLE_RATINGS %>% mutate(rating_cut = cut(as.numeric(averageRating), breaks = 3, labels =
 c("Low", "Mid", "High"))) %>% group_by(rating_cut) %>% summarise(mean_numvotes = mean(as.numeric(numVo
```

```
## # A tibble: 3 x 2
##   rating_cut mean_numvotes
##   <fct>              <dbl>
## 1 Low                 339.
## 2 Mid                 886.
## 3 High               1099.
```

```
name_titles <- NAME_BASICS %>% separate_longer_delim(cols = knownForTitles, delim = ",") %>%
  group_by(primaryName) %>% summarize(titles_per_name = n_distinct(knownForTitles))

ggplot(name_titles) +
  geom_histogram(aes(x = titles_per_name), bins = 50, fill = "plum", color = "black")
```

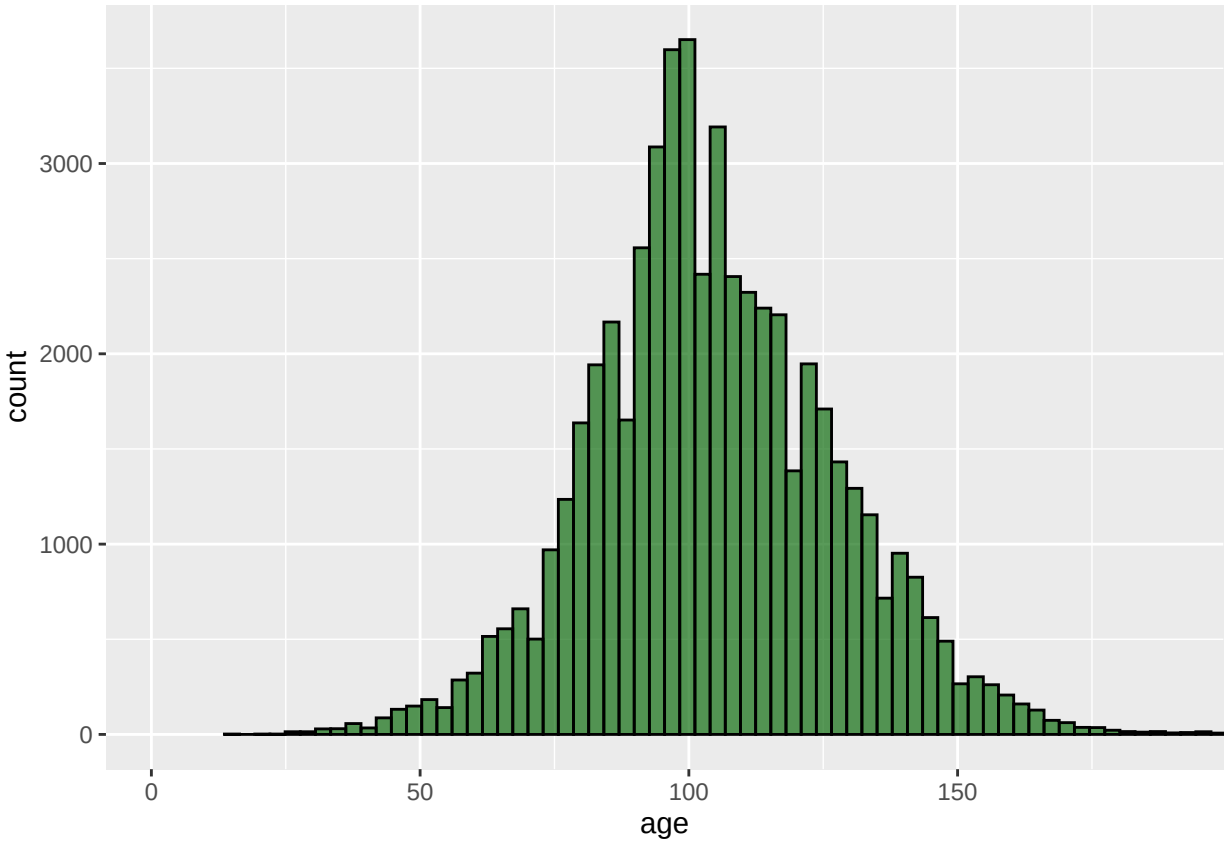



```
living <- NAME_BASICS %>% filter(is.na(deathYear) == TRUE) %>% mutate(age = as.numeric(year(now())) - a
deceased <- NAME_BASICS %>% filter(is.na(deathYear) == FALSE) %>% mutate(age = as.numeric(year(now())) -

ggplot() +
  geom_histogram(data = deceased, aes(x = age),
                 bins = 700, color = "black", fill = "darkgreen", alpha = .65) +
   coord_cartesian(xlim = c(1,190))
```

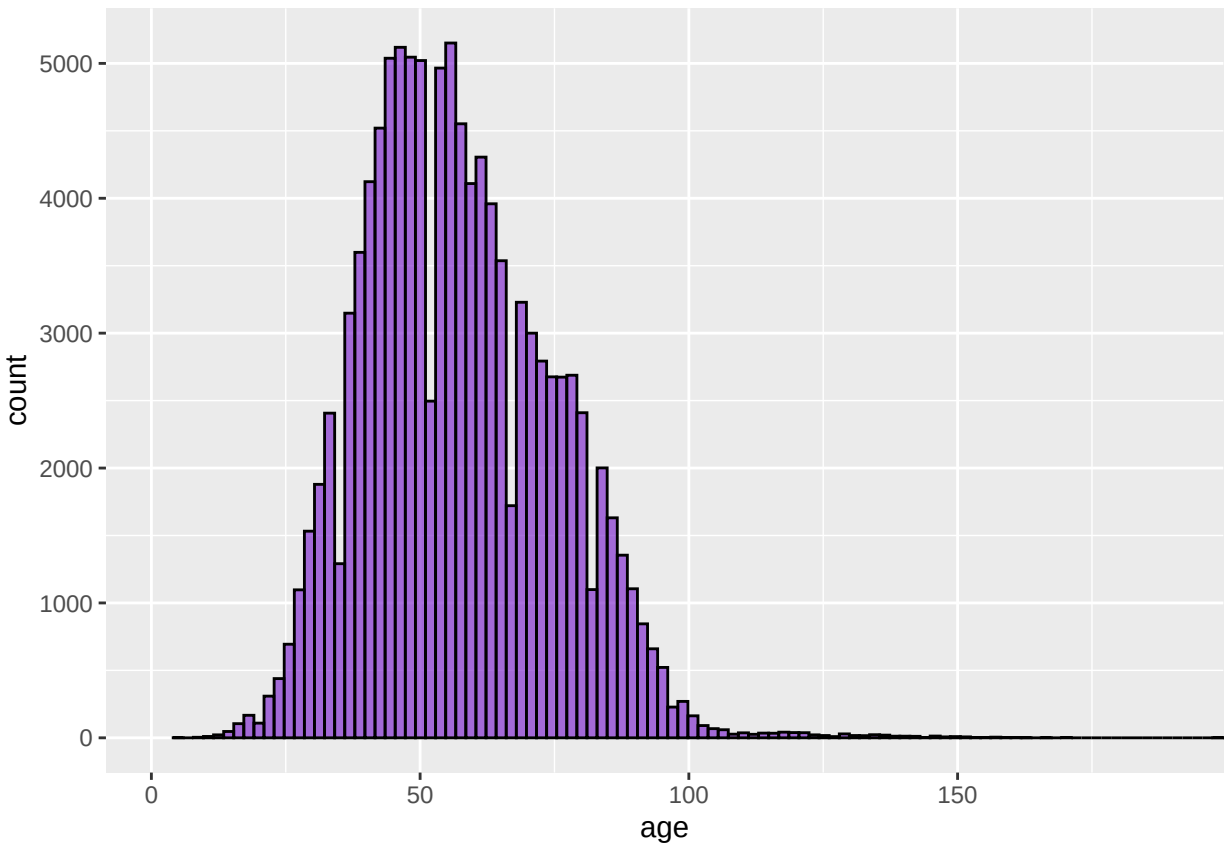
```
## Warning: Removed 1700 rows containing non-finite outside the scale range
```

```
## (`stat_bin()`).
```




```
ggplot() +
  geom_histogram(data = living, aes(x = age),
                 bins = 500, color = "black", fill = "purple3", alpha = .65) +
   coord_cartesian(xlim = c(1,190))
```

```
## Warning: Removed 336069 rows containing non-finite outside the scale range
## (`stat_bin()`).
```



- Find all the actors with first names "Tom", "Thomas", "Thom" or "Tomas". How many are there? 6084
- How many titles use alliteration (i.e., all words in the title start with the same letter)? 2246

```
NAME_BASICS %>% filter(str_detect(primaryName, "Tom|Thomas|Thom|Tomas")) %>% summarize(n())
```

```
##    n()
## 1 6084
```

```
#pattern of letter A-z (), rest of word 0 or more times
#space one or more, detect pattern, rest of word 0 or more times - () must happen at least 1 time
#find end

TITLE_BASICS %>% filter(str_detect(primaryTitle, regex("^([A-Za-z])\\w*(\\s+\\1\\w*)+$", ignore_case = 
```

```
##    n()
## 1 2246
```

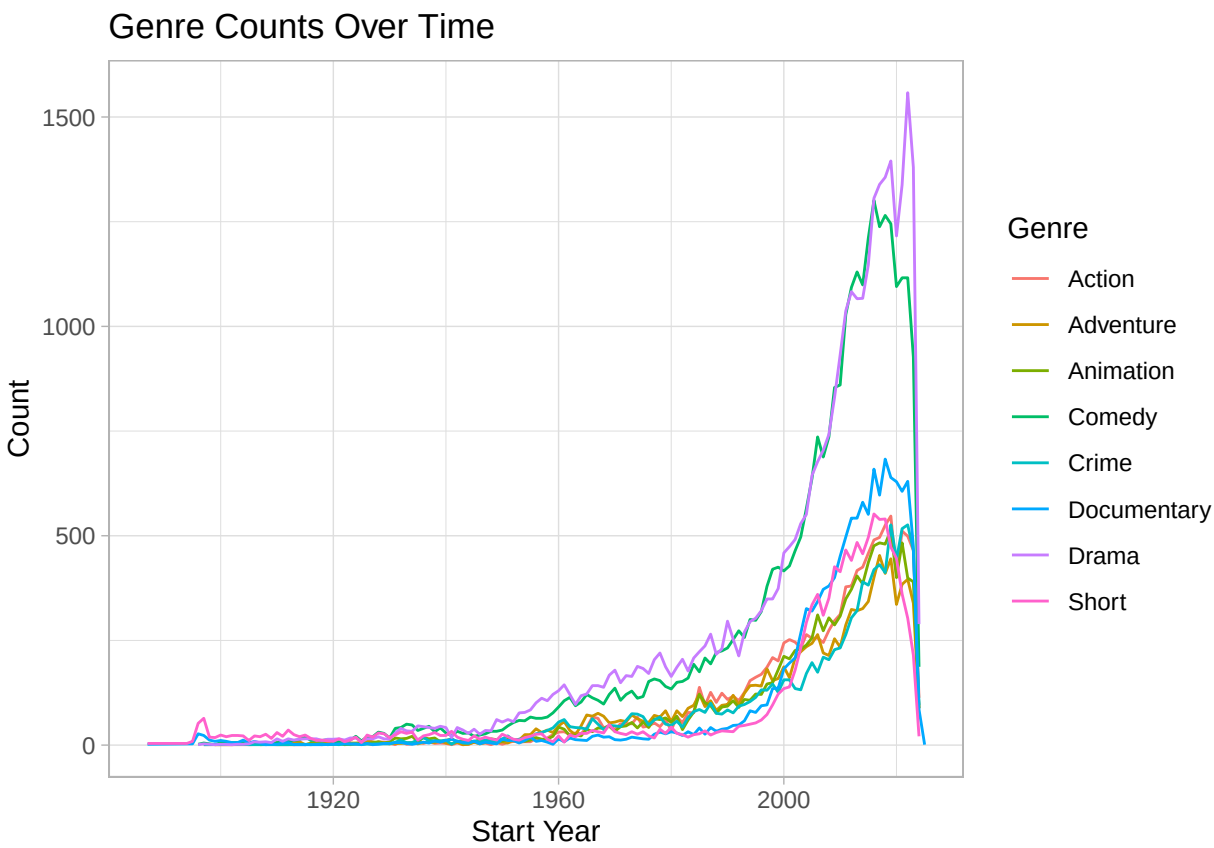
Part III: Pivoting

- Create a new version of the `titles_basics` table that has one row for each title-genre combination. See the `separate_rows` function for a useful too here.
- Using that table, create a line plot of the count different genres over time (you may limit this to the most common genres if you wish).
- Use the `model.matrix` function in the following way: `model.matrix(yourtalltable, ~ genre - 1)` to create a wide table with one column for each genre. Use this table to find the most common pair of genres (hint: use the `cor` function or produce facet plots)
-

```r
#  Create a new version of the `titles_basics` table that has one row for each title-genre combination
TITLE_BASICS_GENRES <- TITLE_BASICS |> separate_rows(genres, sep = ",")

# Line plot of genre counts over time (top 8 genres for readability)
top_genres <- TITLE_BASICS_GENRES |>
  count(genres) |>
  slice_max(n, n = 8) |>
  pull(genres)

TITLE_BASICS_GENRES |>
  filter(genres %in% top_genres, !is.na(startYear)) |>
  count(startYear, genres) |>
  ggplot(aes(x = startYear, y = n, color = genres)) +
  geom_line() +
  theme_light() +
  labs(x = "Start Year", y = "Count", title = "Genre Counts Over Time", color = "Genre")
```




```r
# Use the model.matrix function as described to create a wide table with one column for each genre
# Use this to table to find the most common -air of genres (use the cor function or produce facet plots

GENRE_MATRIX <- model.matrix(~ genres - 1, data = TITLE_BASICS_GENRES)

GENRE_COR <- cor(GENRE_MATRIX)
diag(GENRE_COR) <- NA
GENRE_COR[upper.tri(GENRE_COR)] <- NA

GENRE_COR <- as.data.frame(as.table(GENRE_COR)) |>
```

```r
  filter(!is.na(Freq)) |>
  arrange(desc(Freq))

GENRE_COR |>
  head(10)
```

```
##             Var1            Var2          Freq
## 1       genresNA genresFilm-Noir -3.881425e-05
## 2       genresNA   genresMusical -1.558536e-04
## 3      genresWar        genresNA -1.674222e-04
## 4  genresWestern        genresNA -1.731091e-04
## 5     genresNews        genresNA -1.900577e-04
## 6       genresNA     genresAdult -1.986564e-04
## 7    genresSport        genresNA -2.321527e-04
## 8       genresNA genresBiography -2.338874e-04
## 9       genresNA genresGame-Show -2.536652e-04
## 10  genresSci-Fi        genresNA -2.548495e-04
```

Part IV: Joining Tables

- Join the table with one title-genre per row from the previous section with the ratings table.
  - What is the highest rated genre? What is the lowest rated genre?
  - Using stacked bar charts, investigate the proportions of different genres over time. Are any incresing or decreasing? Use factor functions to help make the plots easier to read.

  Short films and documentaries decreased in proportion over time. Dramas and comedies increased in frequency over time.

- Join the `title_basics` with the ratings table. Have the number of ratings changed over time (based on release year)? Display graphically but also answer with numerical results.

Number of ratings saw a peak around 1940 and some increases centered around 2000.

- Join the names with the ratings and the principals table.
  - Group by individual people, find the top ten people based on the median rating of the titles they appear in.
  - Find the proportions of genres for the the titles that include the top 10 rated principals.
  - Graph ratings against years. What trends do you see? Ratings have increased over time. The best fit line is linear.
- Create a table with one row for each person in the `name_basics` table and title they are known for. Join this to the ratings table to get the ratings of the "known for" films. Find the person (or people) who have the highest median known for rating.
- There are many people with a median rating of 10. Some examples are James Kemp, Giovanni Mammolotti, and Tess Newheart

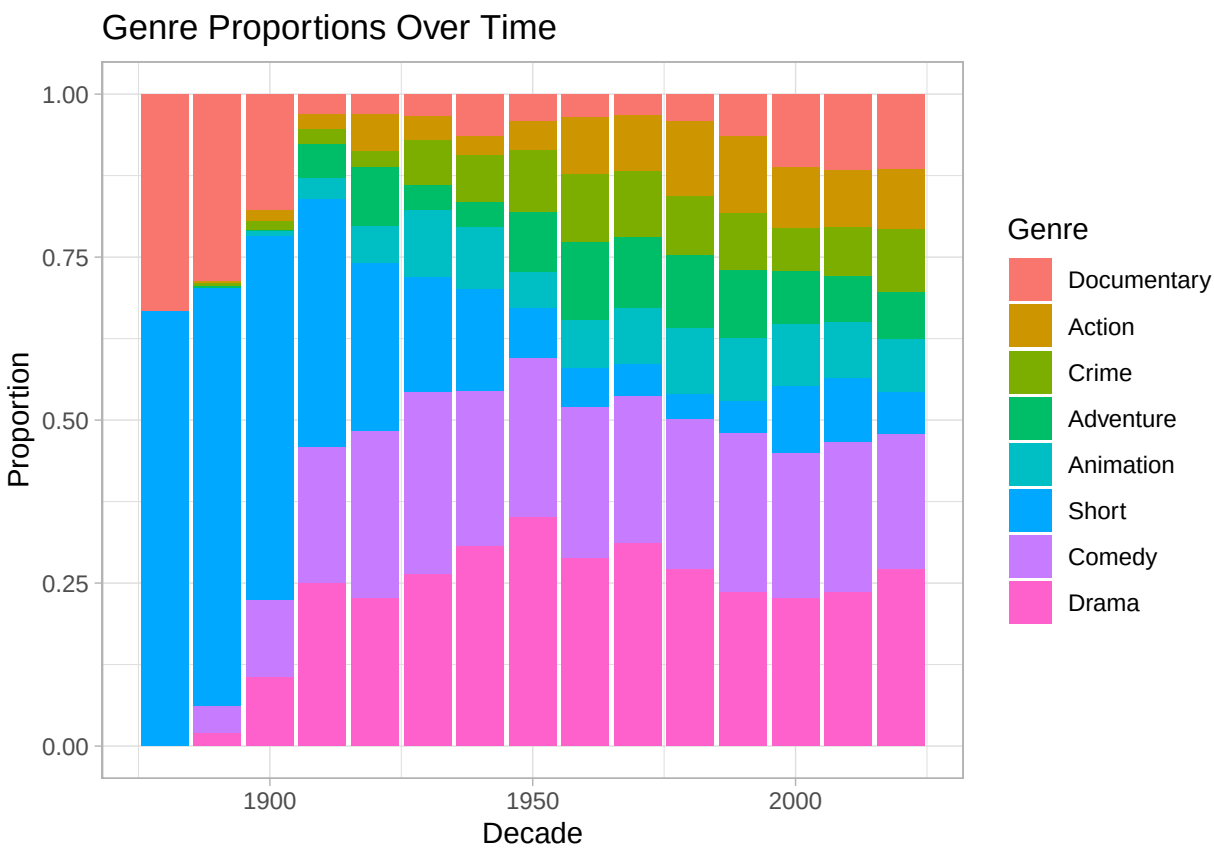
```r
# Join the table with one title-genre per row from the previous section with the ratings table
TITLE_BASICS_GENRES_RATINGS <- TITLE_BASICS_GENRES |>
  left_join(TITLE_RATINGS, by = "tconst")

# Highest and lowest rated genre
TITLE_BASICS_GENRES_RATINGS |>
  group_by(genres) |>
  summarise(mean_rating = mean(averageRating, na.rm = TRUE)) |>
  arrange(desc(mean_rating))
```

```
## # A tibble: 30 x 2
```

```
##    genres       mean_rating
##    <chr>              <dbl>
##  1 History             7.33
##  2 Documentary         7.22
##  3 War                 7.17
##  4 Biography           7.14
##  5 Animation           7.14
##  6 Crime               7.14
##  7 Mystery             7.12
##  8 Adventure           7.12
##  9 Fantasy             7.12
## 10 Family              7.11
## # i 20 more rows
```

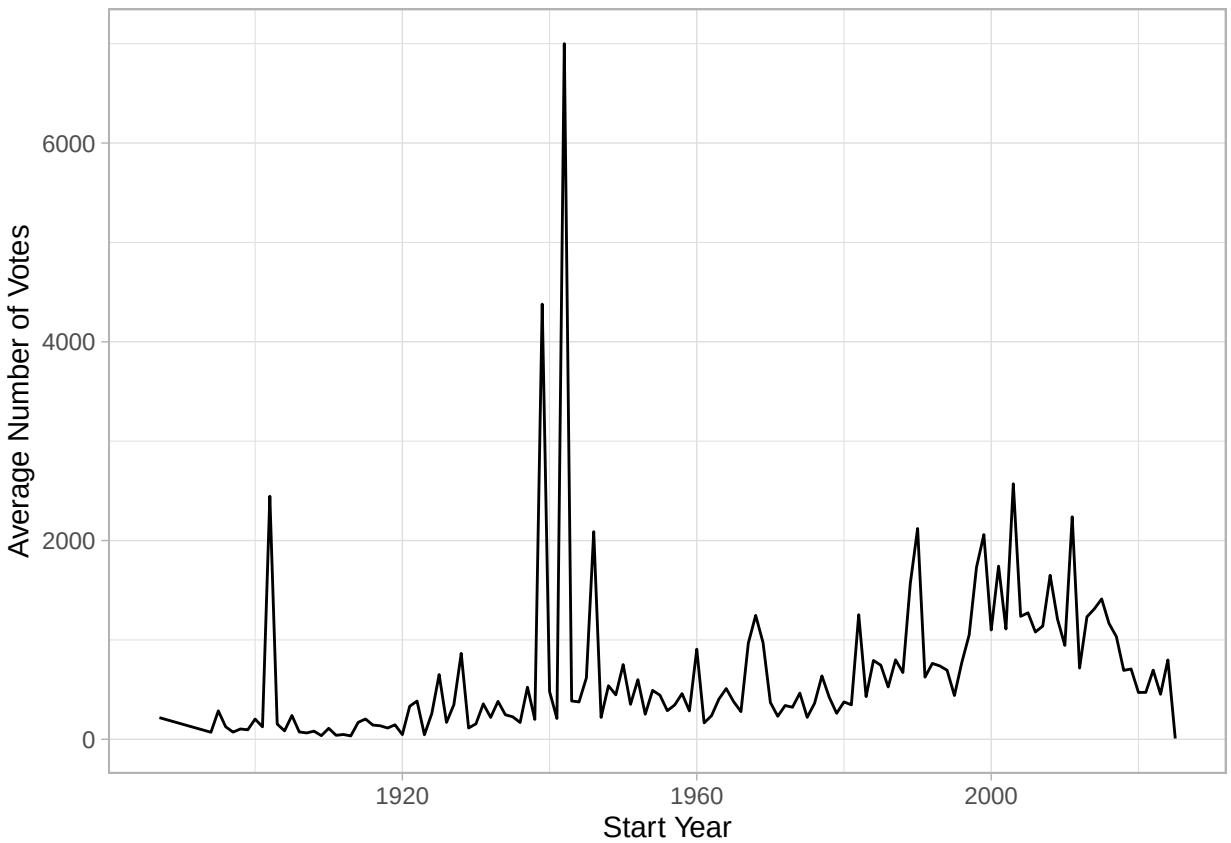
```r
# Stacked bar: genre proportions over time
TITLE_BASICS_GENRES_RATINGS |>
  filter(!is.na(startYear), genres %in% top_genres) |>
  mutate(decade = floor(startYear / 10) * 10) |>
  count(decade, genres) |>
  group_by(decade) |>
  mutate(prop = n / sum(n)) |>
  ggplot(aes(x = decade, y = prop, fill = fct_reorder(genres, prop))) +
  geom_col() +
  theme_light() +
  labs(x = "Decade", y = "Proportion", fill = "Genre",
       title = "Genre Proportions Over Time")
```



```r
# Join the `title_basics` with the ratings table
# Have the number of ratings changed over time (based on release year)?
# Display graphically but also answer with numerical results.
TITLE_BASICS_RATINGS <- TITLE_BASICS |>
  left_join(TITLE_RATINGS, by = "tconst")

TITLE_BASICS_RATINGS |>
  group_by(startYear) |>
  summarise(meanNumVotes = mean(numVotes, na.rm = TRUE)) |>
  ggplot() +
  geom_line(aes(x = startYear, y = meanNumVotes)) +
  theme_light() +
  labs(x = "Start Year", y = "Average Number of Votes")
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).
```




```r
ratings_by_year <- TITLE_BASICS_RATINGS |>
  group_by(startYear) |>
  summarise(meanNumVotes = mean(numVotes, na.rm = TRUE)) |>
  filter(!is.na(startYear))

cor(ratings_by_year$startYear, ratings_by_year$meanNumVotes, use = "complete.obs")
```

```
## [1] 0.3179093
```

```r
# Join the names with the ratings and the principals table
# Group by individual people, find the top ten people based on the median rating of the titles they app
```

```r
# Find the proportions of genres for the the titles that include the top 10 rated principals
# Graph ratings against years
```

```r
PRINCIPALS_RATINGS <- TITLE_PRINCIPALS |>
  left_join(TITLE_RATINGS, by = "tconst") |>
  left_join(TITLE_BASICS |> select(tconst, startYear), by = "tconst")

# Top 10 people by median rating
top10 <- PRINCIPALS_RATINGS |>
  group_by(nconst) |>
  summarise(median_rating = median(averageRating, na.rm = TRUE), .groups = "drop") |>
  slice_max(median_rating, n = 10)

top10 |> left_join(NAME_BASICS |> select(nconst, primaryName), by = "nconst")
```

```
## # A tibble: 843 x 3
##    nconst    median_rating primaryName
##    <chr>             <dbl> <chr>
##  1 nm0010153            10 Adriana Acosta
##  2 nm0012589            10 Daniela Aedo
##  3 nm0014620            10 Liya Ai
##  4 nm0016615            10 Claudia Albertario
##  5 nm0024783            10 Carmen Amezcua
##  6 nm0034922            10 Aristophanes
##  7 nm0037224            10 Ernesto Arreola
##  8 nm0039623            10 Mauricio Aspe
##  9 nm0044021            10 Steven Ayromlooi
## 10 nm0078634            10 Árpád Besenczi
## # i 833 more rows
```

```r
# Genre proportions for top 10 principals
TITLE_BASICS_GENRES |>
  filter(tconst %in% (TITLE_PRINCIPALS |>
    filter(nconst %in% top10$nconst) |>
    pull(tconst))) |>
  count(genres) |>
  mutate(prop = n / sum(n)) |>
  arrange(desc(prop))
```

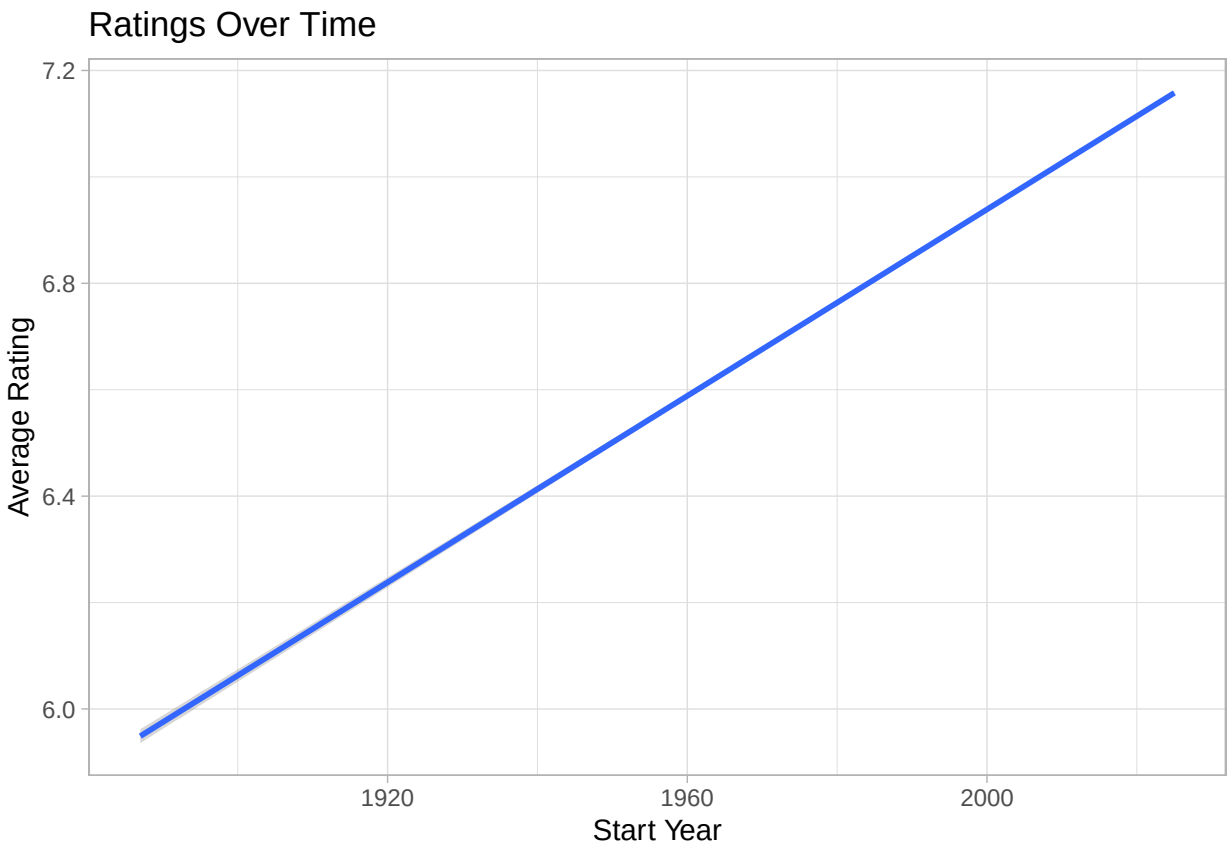
```
## # A tibble: 28 x 3
##    genres          n   prop
##    <chr>       <int>  <dbl>
##  1 Drama         194 0.279
##  2 Comedy         87 0.125
##  3 Documentary    83 0.119
##  4 Short          59 0.0848
##  5 Talk-Show      44 0.0632
##  6 News           35 0.0503
##  7 Family         30 0.0431
##  8 Adventure      20 0.0287
##  9 Music          19 0.0273
## 10 Action         16 0.0230
## # i 18 more rows
```

```r
# Ratings vs years
graph_princ_ratings <- PRINCIPALS_RATINGS |>
  left_join(TITLE_BASICS |> select(tconst, startYear), by = "tconst") |> select(startYear.x, averageRati

  ggplot(data = graph_princ_ratings, aes(x = startYear.x, y = averageRating)) +
  geom_smooth(method = "lm") + theme_light() +
  labs(x = "Start Year", y = "Average Rating",
     title = "Ratings Over Time")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 218 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```




```r
# Create a table with one row for each person in the `name_basics` table and title they are known for. 
NAME_KNOWN_RATINGS <- NAME_BASICS |>
  separate_longer_delim(cols = knownForTitles, delim = ",") |>
  left_join(TITLE_RATINGS, by = c("knownForTitles" = "tconst"))

NAME_KNOWN_RATINGS |>
  group_by(nconst, primaryName) |>
  summarise(median_known_rating = median(averageRating, na.rm = TRUE), .groups = "drop") |>
  slice_max(median_known_rating, n = 5)
```

```
## # A tibble: 259 x 3
##    nconst     primaryName         median_known_rating
##    <chr>      <chr>                             <dbl>
##  1 nm0447300  James Kemp                           10
```

```
##  2 nm0541119  Giovanni Mammolotti                 10
##  3 nm0627886  Tess Newheart                       10
##  4 nm0878577  Stephen Twigger                     10
##  5 nm0971260  Alex Summers                        10
##  6 nm0971408  Lana Cox                            10
##  7 nm0975101  Pepe Pérez                          10
##  8 nm10015015 Raphaël Mondon                      10
##  9 nm10048506 Rico Andriessen                     10
## 10 nm10048510 Sophia Immler                       10
## # i 249 more rows
```

Part V: Profiling and Parallel Processing

- These are large data sets (and yet only a sample of the entire IMDb!), so it make sense spend some time improving our code.
- Pick one or more of the previous problems and profile the performance of that piece. Write up your findings. If you see any opportunities to improve performance, feel fee to implement than and share the results.

```r
library(profvis)
profvis({
  TITLE_BASICS_RATINGS <- TITLE_BASICS |>
  left_join(TITLE_RATINGS, by = "tconst")

TITLE_BASICS_RATINGS |>
  group_by(startYear) |>
  summarise(meanNumVotes = mean(numVotes, na.rm = TRUE))

})
```

```r
profvis({
  TITLE_BASICS_SMALL <- TITLE_BASICS |>
  select(tconst, startYear)
  TITLE_BASICS_RATINGS_SMALL <- TITLE_BASICS_SMALL |>
    left_join(TITLE_RATINGS, by = "tconst")
  TITLE_BASICS_RATINGS_SMALL |>
  group_by(startYear) |>
  summarise(meanNumVotes = mean(numVotes, na.rm = TRUE))
})
```

After running a profile on these functions from part IV, we found that it had a run time of 90ms. The functions perform a left join on the title_basics_ratings and title_basics tables and then summarizes the mean number of votes by release year. The profile flame graph revealed that the left join required 88% of the total run time. A simple fix to improve run time is to first filter the tables to include only the necessary columns for the summary and join. By reducing the number of columns, the run time improves as it processing less data.

- Select a previous computation that could be improved using parallelization and implement a parallelization solution. Using `system.time` show that parallelization improves performance.

```r
library(data.table)
```

```
##
## Attaching package: 'data.table'

## The following objects are masked from 'package:lubridate':
##
```

```
##     hour, isoweek, isoyear, mday, minute, month, quarter, second, wday,
##     week, yday, year

## The following objects are masked from 'package:dplyr':
##
##     between, first, last

## The following object is masked from 'package:purrr':
##
##     transpose
```

```r
library(future.apply)
```

```
## Loading required package: future
```

```r
dt <- as.data.table(PRINCIPALS_RATINGS)
system.time({
  top10 <- PRINCIPALS_RATINGS |>
  group_by(nconst) |>
  summarise(median_rating = median(averageRating, na.rm = TRUE), .groups = "drop") |>
  slice_max(median_rating, n = 10)
})
```

```
##    user  system elapsed
##  16.990   0.029  17.121
```

```r
idx <- split(seq_len(nrow(dt)), sort(rep(1:4, length.out = nrow(dt))))
chunks <- lapply(idx, function(i) dt[i])
chunk_medians <- function(chunk) {
  chunk[, .(median_rating = median(averageRating, na.rm = TRUE)), by = nconst]
}
system.time({
  results_list <- future_lapply(chunks, chunk_medians)
  parallel_medians <- rbindlist(results_list)
})
```

```
##    user  system elapsed
##   2.586   0.153   2.734
```

Parallelization significantly improved the run time for the problem IV function of calculating medians.

- One task we performed involved counting items in strings separated by commas. Propose two different functions that could perform this task. Compare them using bench marking. Which version would you recommend? The first method includes using the str_count function on a fully vectorized input.

```r
count_items_stringr <- function(x) {
  ifelse(is.na(x) | x == "", 0, str_count(x, ",") + 1)
}
```

Another way is to use loops to continually count each comma delimiter.

```r
count_items_loop <- function(x) {
  out <- integer(length(x))
  for (i in seq_along(x)) {
    if (is.na(x[i]) || x[i] == "") {
      out[i] <- 0
    } else {
      out[i] <- length(strsplit(x[i], ",")[[1]])
    }
  }
```

```r
  out
}
test <- paste(rep("a", 1000), collapse = ",")
bm <- bench::mark(count_items_stringr(test), count_items_loop(test))
bm
```

```
## # A tibble: 2 x 6
##   expression                     min   median `itr/sec` mem_alloc `gc/sec`
##   <bch:expr>                <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
## 1 count_items_stringr(test)   45.6us   48.6us    17965.      19KB        0
## 2 count_items_loop(test)     148.3us  155.2us     5725.    56.9KB        0
```

I would recommend the vectorized version as it is much more efficient in counting the number of items.

Part VI: Shiny Applications

Application 1

Using results from the previous section, create a shiny application that allows users to interact with the with the IMDb data. The application should use both interactive graphs and at least 3 widgets.

```r
library(shiny)
library(tidyverse)


principals <- read_rds("../data/title_principals.rda")
names_tbl  <- read_rds("../data/name_basics.rda")
ratings    <- read_rds("../data/title_ratings.rda")
basics     <- read_rds("../data/title_basics.rda")


joined <- principals |>
  left_join(ratings, by = "tconst") |>
  left_join(basics |> select(tconst, startYear, primaryTitle, genres), by = "tconst")

top10_genres <- joined$genres |>
  str_split(",") |>
  unlist() |>
  tibble(genre = _) |>
  count(genre, sort = TRUE) |>
  slice_head(n = 10) |>
  pull(genre)

joined_top10 <- joined |>
  filter(
    map_lgl(genres, ~ any(str_split(.x, ",")[[1]] %in% top10_genres))
  )

ui <- fluidPage(
  titlePanel("IMDb Explorer"),
  sidebarLayout(
    sidebarPanel(
      selectInput("category", "Job Category:", choices = unique(joined_top10$category)),
      sliderInput("year", "Start Year:", min = 1900, max = 2025, value = c(1950, 2025)),
      checkboxGroupInput("genres", "Genres: ", choices = top10_genres, selected = top10_genres)
```

```r
    ),
    mainPanel(
      plotOutput("genrePlot"),
      plotOutput("ratingPlot", click = "plot_click"),
      verbatimTextOutput("click_info")
    )
  )
)

server <- function(input, output) {

  filtered <- reactive({
    joined_top10 |>
      filter(category == input$category,
             !is.na(startYear),
             startYear >= input$year[1],
             startYear <= input$year[2] )|>
             filter(
               map_lgl(genres, ~ any(str_split(.x, ",")[[1]] %in% input$genres))
             )


  })

  output$ratingPlot <- renderPlot({
    ggplot(
      filtered(), aes(startYear, averageRating, color = averageRating)) +
      geom_point(alpha = 0.4, size = 2) +
      geom_smooth(se = FALSE, color = "purple", linewidth = 1) +
      scale_color_viridis_c(option = "plasma") +
      labs(
        title = "Ratings Over Time",
        x = "Release Year",
        y = "Average IMDb Rating",
        color = "Rating"
    )
  })
  output$genrePlot <- renderPlot({
    ggplot(filtered(), aes(category)) +
      geom_bar() +
      labs(
        title = "Number of Titles by Job Category",
        x = "Category",
        y = "Count"
      )
  })


  output$click_info <- renderPrint({
    req(input$plot_click)
    nearPoints(filtered(), input$plot_click, maxpoints = 1)
  })
}
```

```r
shinyApp(ui = ui, server = server)
```

Application 2

In the principals table, there is a `category` column. Use this column as a primary filter to allow users to then select specific job categories. After select the specific job categories, display information from another table.

```r
# In the principals table, there is a `category` column. Use this column as a primary
# filter to allow users to then select specific job categories. After select the specific
# job categories, display information from another table.

library(shiny)
library(tidyverse)

# Load the correct .rda files
principals <- read_rds("../data/title_principals.rda")
names_tbl  <- read_rds("../data/name_basics.rda")
ratings    <- read_rds("../data/title_ratings.rda")

# Define UI for application
ui <- fluidPage(
  titlePanel("Job Categories"),
  sidebarLayout(
    sidebarPanel(
      selectInput("category", "Select Job Category:", choices = NULL)
    ),
    mainPanel(

      h4("People in this category (with title ratings)"),
      tableOutput("job_info")
    )
  )
)


# Define server logic

server <- function(input, output, session) {


  # Populate dropdown from actual category column
  observe({
    categories <- sort(unique(principals$category))
    updateSelectInput(session, "category", choices = categories)
  })

  output$job_info <- renderTable({
    req(input$category)

    # Filter principals by selected category, then join names + ratings
    principals |>
      filter(category == input$category) |>
      left_join(names_tbl |> select(nconst, primaryName), by = "nconst") |>
      left_join(ratings, by = "tconst") |>
```

```r
      select(primaryName, tconst, category, characters, averageRating, numVotes) |>
      arrange(desc(averageRating)) |>
      head(50)  # limit rows for display
  })
}


# Run the application
shinyApp(ui = ui, server = server)
```

Extra Credit: 6 Degrees of Kevin Bacon

Create an app to allow users to play Six Degrees of Kevin Bacon.

Create a Shiny application where a person can type the primary title of movie or TV show. Then have app show all the people who had a role in the show. Let the user select a person in that cast and show all other people who have been in a title with that person. Repeat up to 6 times. If "Kevin Bacon" (`nconst == 'nm0000102'`) ever appears in the list, let the player know they have won! If they click more than 6 times, let them know they have lost.

```r
library(shiny)
library(tidyverse)

# ---- Data & Helpers (do not modify) ----

movie_actors <- read_csv("movie_actors.csv", show_col_types = FALSE)

# Lookup tables for fast access
actor_to_movies <- movie_actors |>
  group_by(actor) |>
  summarise(movies = list(movie), .groups = "drop") |>
  deframe()

movie_to_actors <- movie_actors |>
  group_by(movie) |>
  summarise(actors = list(actor), .groups = "drop") |>
  deframe()

non_bacon_actors <- setdiff(sort(unique(movie_actors$actor)), "Kevin Bacon")

# Sampling weights: proportional to number of movies (favors recognizable actors)
actor_weights <- lengths(actor_to_movies)[non_bacon_actors]

# Helper: get movies for an actor, sorted by release year (extracted from the
# "Title (YYYY)" suffix)
get_movies <- function(actor_name) {
  m <- actor_to_movies[[actor_name]]
  years <- as.integer(str_extract(m, "(?<=\\()\\d{4}(?=\\)$)"))
  m[order(years, m)]
}

# Helper: get cast for a movie
get_cast <- function(movie_name) {
  sort(movie_to_actors[[movie_name]])
```

```r
}

# ---- UI (do not modify) ----

ui <- fluidPage(
  titlePanel("Six Degrees of Kevin Bacon"),
  sidebarLayout(
    sidebarPanel(
      actionButton("new_game", "New Game"),
      hr(),
      textOutput("step_display"),
      hr(),
      radioButtons("selection", "Make a choice:", choices = c("Click 'New Game' to start")),
      actionButton("choose", "Select")
    ),
    mainPanel(
      h4("Your Path"),
      htmlOutput("path_display"),
      hr(),
      h3(textOutput("result_message"))
    )
  )
)









# ---- Server ----

server <- function(input, output, session) {

  # Game state - use these reactive values in your TODOs below.
  # - active: TRUE while a game is in progress
  # - phase: "actor" when the player is choosing an actor,
  #          "movie" when choosing a movie
  # - step: number of actors selected so far (0 to 6)
  # - path: character vector of the trail, alternating actor/movie names
  # - visited: character vector of actors already chosen (to exclude from casts)
  # - visited_movies: character vector of movies already chosen (to exclude
  #   from future movie option lists)
  # - won / lost: flags for game outcome
  game <- reactiveValues(
    active = FALSE,
    phase  = "actor",
    step   = 0L,
    path   = character(),
    visited = character(),
    visited_movies = character(),
```

```r
    won    = FALSE,
    lost   = FALSE
  )

  # Step counter display (do not modify)
  output$step_display <- renderText({
    if (!game$active && !game$won && !game$lost) return("")
    paste0("Step ", game$step, " of 6")
  })

  # ============================================================
  # TODO 1: New Game button
  # ============================================================
  # When input$new_game is clicked:
  #   - Set game$active to TRUE, game$won and game$lost to FALSE
  #   - Reset game$step to 0, game$path, game$visited, and game$visited_movies
  #     to empty character()
  #   - Set game$phase to "actor"
  #   - Sample 5 actors from non_bacon_actors with probability proportional
  #     to their number of movies. Pass prob = actor_weights to sample().
  #   - Use updateRadioButtons() to set those actors as the choices for
  #     the "selection" input. The syntax is:
  #       updateRadioButtons(session, "selection", choices = your_vector)
observeEvent(input$new_game, {
  game$active <- TRUE
  game$won <- FALSE
  game$lost <- FALSE
  game$step <- 0L
  game$path <- character()
  game$visited <- character()
  game$visited_movies <- character()
  game$phase <- "actor"

  sampled_actors <- sample(non_bacon_actors, size = 5, prob = actor_weights)
  updateRadioButtons(session, "selection", choices = sampled_actors)
})


  # ============================================================
  # TODO 2: Select button (main game logic)
  # ============================================================
  # When input$choose is clicked:
  #   Use req(game$active, input$selection) to guard the handler.
  #
  #   If game$phase == "actor":
  #     - Add the chosen actor (input$selection) to game$path and game$visited
  #     - Increment game$step by 1
  #     - If the chosen actor is "Kevin Bacon": set game$won to TRUE,
  #       game$active to FALSE, and return()
  #     - If game$step >= 6: set game$lost to TRUE, game$active to FALSE,
  #       and return()
  #     - Otherwise: get the actor's movies using get_movies(), remove any
  #       already-visited movies with setdiff(movies, game$visited_movies),
```

```r
#       update the radio buttons with updateRadioButtons(), and set
#       game$phase to "movie". (Note: get_movies() already returns movies
#       sorted by release year, so the options appear in chronological order.)
#
#   If game$phase == "movie":
#     - Add the chosen movie (input$selection) to game$path AND to
#       game$visited_movies
#     - Get the cast using get_cast()
#     - Remove already-visited actors with setdiff(cast, game$visited)
#     - Update the radio buttons with the filtered cast
#     - Set game$phase to "actor"
observeEvent(input$choose, {
  req(game$active)
  req(input$selection)
  req(nchar(input$selection) > 0)

  if(game$phase == "actor") {
    chosen_actor <- input$selection
    game$path <- c(game$path, chosen_actor)
    game$visited <- c(game$visited, chosen_actor)
    game$step <- game$step + 1L

    if(chosen_actor == "Kevin Bacon") {
      game$won <- TRUE
      game$active <- FALSE
      updateRadioButtons(session, "selection", choices = c("Game over"))
      return()
    }

    if(game$step >= 6) {
      game$lost <- TRUE
      game$active <- FALSE
      updateRadioButtons(session, "selection", choices = c("Game over"))
      return()
    }

    movies <- get_movies(chosen_actor)
    movies <- setdiff(movies, game$visited_movies)
    updateRadioButtons(session, "selection", choices = movies)
    game$phase <- "movie"


  } else if(game$phase == "movie") {
    req(game$active)
    chosen_movie <- input$selection
    game$path <- c(game$path, chosen_movie)
    game$visited_movies <- c(game$visited_movies, chosen_movie)

    cast <- get_cast(chosen_movie)
    cast <- setdiff(cast, game$visited)

    if ("Kevin Bacon" %in% get_cast(chosen_movie) && !"Kevin Bacon" %in% cast) {
      cast <- c(cast, "Kevin Bacon")
```

```r
    }

    updateRadioButtons(session, "selection", choices = cast)
    game$phase <- "actor"
  }
})


# ============================================================
# TODO 3: Path display
# ============================================================
# Render the path as formatted HTML using renderUI().
#   - If game$path is empty, show: tags$p(tags$em("No moves yet."))
#   - Otherwise, loop through game$path:
#       - Odd-indexed entries are actors: wrap in tags$strong()
#       - Even-indexed entries are movies: wrap in tags$em()
#     Separate entries with arrows: HTML(" &rarr; ")
#   - Combine everything with do.call(tagList, ...)
output$path_display <- renderUI({
  if(length(game$path) == 0) {
    return(tags$p(tags$em("No moves yet.")))
  }

  combined_path <- list()

  for(i in seq_along(game$path)) {
    if(i %% 2 == 1) {
      combined_path <- c(combined_path, list(tags$strong(game$path[i])))
    } else {
      combined_path <- c(combined_path, list(tags$em(game$path[i])))
    }

    if(i < length(game$path)) {
      combined_path <- c(combined_path, list(HTML(" &rarr; ")))
    }
  }

  do.call(tagList, combined_path)
})

# ============================================================
# TODO 4: Result message
# ============================================================
# Use renderText() to show:
#   - If game$won: a message like "You found Kevin Bacon in X steps!"
#   - If game$lost: "Game over! You didn't reach Kevin Bacon in 6 steps."
#   - Otherwise: "" (empty string)
output$result_message <- renderText({
  if(game$won) {
    paste0("You found Kevin Bacon in ", game$step - 1, " steps!")
  } else if(game$lost) {
    "Game over! You didn't reach Kevin Bacon in 6 steps."
  } else {
```

```
      ""
    }
  })
}

shinyApp(ui, server)
```